

Introduction to Angular and Angular CLI

What is Angular?

Angular is an open-source, front-end web application framework used to build:

- Single Page Applications (SPA)
- Dynamic and interactive web applications
- Large-scale, enterprise-level web systems

Angular applications run in the browser, but are developed using TypeScript.

Why Angular is Used?

Angular is used because it provides:

1. Component-based architecture
2. Fast performance
3. Automatic UI updates
4. Built-in routing
5. Form handling and validation
6. Strong structure for large projects

Key Features of Angular

Component-Based Architecture

- Application is divided into small reusable components
- Each component has:
 - Logic (.ts)
 - View (.html)
 - Style (.css)

Single Page Application (SPA)

- Only one HTML page loads initially
- Page content updates without reloading
- Faster and smoother user experience

3 Data Binding

Angular connects data and UI automatically.

Types:

- Interpolation `{{ }}`
- Property Binding `[ ]`
- Event Binding `( )`
- Two-Way Binding `[ ( ) ]`

Directives

Directives control the DOM.

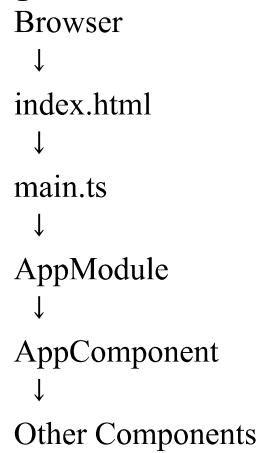
Examples:

- `*ngIf` – conditional display
- `*ngFor` – looping
- `ngClass`, `ngStyle`

TypeScript Support

- Strong typing
- Error detection at compile time
- Better code maintainability

Angular Architecture (High Level)



What is Angular CLI?

Angular CLI (Command Line Interface) is a tool used to:

- Create Angular projects
- Generate components, services, modules
- Run and test applications
- Build applications for deployment

CLI automates repetitive tasks.

Why Angular CLI is Important?

Without CLI:

- Manual folder creation
- Manual configuration
- Error-prone setup

With CLI:

- One command → full project
- Standard structure
- Faster development
- Industry best practices followed

Installing Angular CLI

Step 1: Install Node.js

(Node.js is required to run Angular)

Step 2: Install Angular CLI

`npm install -g @angular/cli`

Step 3: Verify Installation

`ng version`

Creating an Angular Project Using CLI

ng new my-angular-app

CLI automatically:

- Creates folders
- Installs dependencies
- Configures TypeScript
- Sets up build system

Running Angular Application

cd my-angular-app

ng serve

✓ Application runs at

http://localhost:4200

Common Angular CLI Commands

Command	Purpose
ng new app-name	Create new project
ng serve	Run application
ng generate component name	Create component
ng generate service name	Create service
ng build	Build project
ng test	Run tests

What Happens When **ng serve** is Executed?

1. TypeScript → JavaScript compilation
2. Angular bundles files
3. Development server starts
4. Browser loads the app
5. Changes auto-reload

step by step in Visual Studio Code.

Check your project folder

You may have `app.ts` instead of `app.component.ts`.

Check these files inside `src/app`

- `app.ts`
- `app.html`
- `app.component.ts`
- `app.component.html`

Open `src/app/app.ts`.

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-root',  
  template: `  
    <h1>Welcome to Angular</h1>  
    <p>This is my first Angular program.</p>  
  `,  
})  
export class App {  
}
```

Run the program: `ng serve`

Open Chrome:

`http://localhost:4200`

Expected output:

Welcome to Angular

This is my first Angular program.

Angular CLI Program

In Visual Studio Code:

1. Open the `angular-demo` folder.
2. Click Terminal → New Terminal.
3. Make sure the terminal is inside the `angular-demo` folder.

Generate a component using Angular CLI

In the terminal, type: **ng generate component student**
Or use the short command: **ng g c student**

What Angular CLI does
It automatically creates a new component.

In newer Angular versions, you may see:

```
src/app/student/  
├── student.ts  
├── student.html  
├── student.css  
└── student.spec.ts
```

In older versions, you may see:

```
src/app/student/  
├── student.component.ts  
├── student.component.html  
├── student.component.css  
└── student.component.spec.ts
```

This automatic file creation is the Angular CLI part of the program.

Open the generated student file

In the VS Code Explorer:

src → app → student

Open the TypeScript file.

It may be named:

- **student.ts**, or
- **student.component.ts**

You should see generated code similar to:

Replace the code in your student TypeScript file with:

student.ts

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-student',  
  template: `  
    <h2>Student Information</h2>  
    <p>Name: Lakshmi</p>  
    <p>Department: CSE (AI & ML)</p>  
    <p>College: CBIT</p>  
  `,  
})  
export class Student {
```

```
}
```

app.ts

Replace its code with:

```
import { Component } from '@angular/core';
```

```
import { Student } from './student/student';
```

```
@Component({
```

```
  selector: 'app-root',
```

```
  imports: [Student],
```

```
  template: `
```

```
    <h1>Angular CLI Demonstration</h1>
```

```
    <app-student></app-student>
```

```
`
```

```
})
```

```
export class App {
```

```
}
```

Run the application: ng serve

Open the browser:

<http://localhost:4200>

OUTPUT:

Angular CLI Demonstration

Student Information

Name: Lakshmi

Department: CSE (AI & ML)

College: CBIT

How to show the difference in your practical

Step	Angular	Angular CLI
Create application	Write Angular components and templates.	ng new angular-demo

Create student component	Write the component code.	<code>ng g c student</code> automatically creates files.
Modify component	Use TypeScript and HTML.	Use generated files and Angular code.
Run application	Angular application displays output.	<code>ng serve</code> starts the development server.